

Assignment 1 — From “It Trains” to “I Know Why It’s Slow”

Covers: Week 1 (Fundamentals of AI/ML) + Week 2 (HPC Fundamentals & Profiling) **Effort budget:** ~3 hours total, all-inclusive (code + report) **Deliverables:** 2–3 original code files you write yourself + 1 short report (PDF or Markdown, 1.5–2 pages) **Tools needed:** Python 3.8+, NumPy, a C compiler (clang/gcc). No GPU, no PyTorch, no internet access required.

Ground rule: write your own code. This assignment asks you to *implement* the Week 1/2 concepts from first principles, not to run or patch the course’s example scripts. Do not open or copy from `examples/week1_ml_foundations/` or `examples/week2_profiling/` while completing Parts A–C — work from the slide equations and the spec below instead. Those example scripts are there for you to sanity-check your own numbers **afterward** (see Part C, step 4), not to use as a starting point.

Why this assignment

Week 1 ended with a claim: a neural net is just GEMMs + a nonlinearity + a loss + gradient descent, and at real model sizes that workload overflows one machine on **compute, memory, or I/O**. Week 2 gave you the tools to *prove* claims like that with numbers instead of vibes: the memory hierarchy, arithmetic intensity, the roofline model, and the course’s core discipline —

measure → classify → change one thing → re-measure

The only way to know you actually understand the forward pass / backward pass / SGD update — or the cache/roofline arguments — is to write the code that implements them yourself, with nothing to copy from. That’s what this assignment tests.

Part A — Build a neural net from scratch (75 min)

File to submit: `mlp_from_scratch.py`

Implement a 2-layer MLP ($2 \rightarrow h \rightarrow 1$) in pure NumPy — **no autograd, no PyTorch/TensorFlow, no sklearn** — trained with plain SGD on a toy non-linear binary classification problem.

1. **Data** (you may use this generator verbatim — data generation isn’t the learning objective here):

```
import numpy as np
def make_xor_blobs(n=400, noise=0.25, seed=0):
    rng = np.random.default_rng(seed)
    centers = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
    labels = np.array([0,1,1,0]) # XOR pattern -> not linearly separable
    idx = rng.integers(0, 4, size=n)
    X = centers[idx] + rng.normal(scale=noise, size=(n, 2))
    y = labels[idx].reshape(-1, 1).astype(float)
    return X, y
```

2. **You must write, yourself:**

- `forward(X, W1, b1, W2, b2)` — two GEMMs + a nonlinearity (your choice: ReLU or tanh) on the hidden layer, sigmoid on the output.
 - A loss function — binary cross-entropy or MSE, your choice, plus the scalar loss value each epoch.
 - `backward(...)` — the gradients of the loss w.r.t. `W1`, `b1`, `W2`, `b2`, derived by hand via the chain rule. (You may derive on paper first; put a one-line comment above each gradient line stating which chain-rule term it is.)
 - A plain SGD update loop: `W -= lr * grad`, no momentum/Adam/optimizer library.
3. **Split** your data 80/20 into train/validation. Train for enough epochs to see train accuracy plateau above 90%.
 4. **Reproduce the overfitting curve:** re-run with a much smaller training set (e.g. 40 points) and many epochs, and show training loss keeps falling while validation loss turns back up. Save both loss curves (matplotlib PNG if available; otherwise print a loss value every N epochs so the trend is visible as a table — either is fine).

Record for the report: final train/val loss and accuracy for both runs (the plateauing run and the overfitting run), and your two loss curves (plots or printed tables).

Part B — Write your own training-cost estimator (20 min)

File to submit: `training_cost_estimator.py`

Don't run the course's calculator — write a short script (~20–30 lines) from the two back-of-envelope laws in the Week 1 slides:

- Compute per training step: $6 \cdot P \cdot T$ FLOPs (P = parameter count, T = training tokens)
- Optimizer memory: $16 \cdot P$ bytes (FP16 weights + grads + FP32 Adam state)

Your script should: 1. Take P and T as inputs (hardcoded variables or `argparse` — your choice). 2. Print total training FLOPs and optimizer memory (convert to GB). 3. Given a GPU's peak throughput (pick a real published number, e.g. an A100's ~312 TFLOP/s bf16 figure, and cite where you got it) and an assumed utilization (e.g. 40%), compute and print the wall-clock time to train on **1 GPU** and on **1,024 GPUs** (assume perfect linear scaling for this estimate — you'll learn in Week 4 why that's optimistic).

Run it for **one real model you choose** that is not GPT-2/BERT/GPT-3/LLaMA from the slides (e.g. Mistral-7B, Llama-3-8B, Phi-3, or any model you're curious about). Estimate P and T yourself — order-of-magnitude is fine, cite where the estimate came from.

Record for the report: your model choice, P , T , the three computed numbers, and which of the **three walls** (compute / memory / I/O) it hits first, with a one-sentence justification.

Part C — Write and run your own micro-benchmark (60 min)

Pick **exactly one** of the two options below. Write the benchmark yourself, run the full **measure** → **classify** → **change one thing** → **re-measure** loop, and only then compare against the course's reference script as a sanity check.

Option 1 — Cache locality (C)

File to submit: `cache_locality.c`

Write a C program that: 1. Allocates an $N \times N$ matrix of doubles (start with $N = 4096$). 2. Sums every element twice: once traversing row-major (stride 1, inner loop over columns) and once traversing column-major (stride N , inner loop over rows). 3. Times both loops (`clock_gettime(CLOCK_MONOTONIC, ...)` or equivalent) and prints both times and the slowdown ratio.

Option 2 — Roofline (Python + NumPy)

File to submit: `roofline_bench.py`

Write a script that: 1. Implements **SAXPY** ($y = a*x + y$) and **GEMM** ($C = A @ B$) benchmarks using NumPy arrays. 2. Times each with a proper warmup (a few untimed calls first) then averages over multiple timed iterations — the “measure, don’t guess” recipe from the slides. Don’t time a single cold call. 3. Computes, for each kernel: FLOPs performed (SAXPY: 2 per element; GEMM: $2N^3$), bytes moved (SAXPY: ~ 24 bytes/element; GEMM: $\sim 3N^2 \cdot 8$ bytes), arithmetic intensity $AI = \text{FLOPs} / \text{bytes}$, and achieved GFLOP/s. 4. Prints a classification (memory-bound vs compute-bound) for each kernel based on its AI.

Then, regardless of which option you picked:

3. **Measure:** run your benchmark at the default size and record the numbers.
4. **Classify:** state which kernel/access-pattern is memory-bound vs compute-bound (or, for cache locality, which layout wins), and *why*, in one sentence tied to the memory hierarchy / roofline theory.
5. **Change one thing:** increase the problem size (bigger N , bigger vector length) — for cache locality this should widen the gap once the matrix stops fitting in cache; for roofline, a bigger GEMM should push AI and achieved GFLOP/s higher.
6. **Re-measure:** run again, record the new numbers, and state whether the change moved in the direction the theory predicted.
7. **Sanity check:** *now* run the course’s matching demo (`examples/week2_profiling/cache_locality.c` or `roofline.py`) and note whether your numbers are in the same ballpark. If they’re wildly different, say what you think explains the gap (compiler flags, allocator behavior, NumPy’s BLAS backend, etc.) — this is a legitimate and expected part of the exercise, not a sign you did it wrong.

Part D — Short report (25 min)

Write **1.5–2 pages** (Markdown or PDF) with these sections:

1. **Part A summary** — your architecture choices (activation, loss), final numbers for both runs, and one sentence on why the overfitting run matches (or doesn’t) the bias-variance curve from the slides.
2. **Part B summary** — your chosen model, P , T , the three key numbers, and which wall it hits first.

3. **Part C summary** — your baseline numbers, the change you made, the re-measured numbers, whether the delta matched theory, and the result of your sanity check against the course’s reference script.
 4. **Synthesis (the actual point of this assignment)** — in 3–5 sentences, connect Parts A–C: using the model from Part B and the vocabulary from Part C (arithmetic intensity, memory-bound vs compute-bound, cache locality), argue what a *first* optimization step would be if you actually had to train that model, and what evidence you’d want to collect before doing anything else. There’s no single correct answer — you’re graded on whether the argument is evidence-based.
-

Submission

- `mlp_from_scratch.py`
- `training_cost_estimator.py`
- `cache_locality.c` or `roofline_bench.py` (whichever option you chose)
- `report.md` or `report.pdf`

Rubric (100 pts)

Component	Points	What’s being checked
Part A — forward/backward/SGD written from scratch and correct (loss decreases, no autograd used)	25	Code is original, runs, and the math is right
Part A — overfitting run reproduces train/val divergence	10	The curve actually diverges as predicted
Part B — estimator implements 6PT / 16P correctly + reasonable model choice	15	Formulas correct; wall identification matches the numbers
Part C — benchmark written from scratch with correct warmup/timing methodology	15	No single cold-call timing; averaged over iterations
Part C — full measure→classify→change→re-measure cycle + sanity check	20	Cycle complete; delta interpreted against theory; comparison against reference script done
Synthesis section	10	Argument uses evidence from A–C rather than restating slide content
Clarity / concision (report stays within ~2 pages)	5	—